

SANTANDER X · QUANTUM AI LEAP · 2026

EmpowerTours Quantum Portfolio

Q-Day-resistant DeFi, shipped today.

QAOA on IBM Heron · Hedged post-quantum signatures · On-chain
provenance on Monad

THE THREAT

Every ECDSA signature you publish today is a hostage to tomorrow's QPU.

Harvest now. Decrypt later. The adversary already has your 2026 trade orders — they just haven't broken the curve yet.

WHY DEFI GETS HIT FIRST

Public chains are an open archive of signatures.

- Every transaction broadcasts an **ECDSA signature** in the clear.
- Indexers store them **forever**.
- A cryptographically-relevant QPU forges any one of them.
- The chain doesn't know which.

TODAY'S DEFENSE

None. Most wallets still sign with secp256k1.

Most "post-quantum DeFi" decks are **roadmaps**.

THE STACK

Three layers. Each one survives the other two.

1. ALLOCATE

QAOA on IBM

Heron QPU picks
Markowitz-optimal
weights under risk
constraints.

2. SIGN

Hedged PQ: every
order carries ML-
DSA-65 + SLH-
DSA-SHAKE-256s +
Ed25519. **Any one**
survives.

3. ANCHOR

SHA-256(order)

committed to
Monad's
AuditAnchor before
the vault will
execute it.

*One layer broken $\neq$ system broken. The threat model is **every algorithm***

THE QUANTUM LAYER

QAOA on real hardware, reported with statistical honesty.

- Circuit compiled + queued against **IBM Heron** via Qiskit Runtime.
- We report **mean $\pm$ stdev across 12 seeds** vs SLSQP baseline.
- Not single-best-run cherry-picking — the failure mode of

WHAT THIS IS, HONESTLY

QAOA is **not** yet faster than SLSQP for this problem size. We measure the **gap** as it closes. Honest measurement is the deliverable.

THE SIGNATURE LAYER

Don't bet on one algorithm. Hedge the bet.

ALGORITHM	FAMILY	WHY INCLUDE
ML-DSA-65	Lattice (FIPS 204)	First quantum-resistant choice.
SLH-DSA-SHAKE-256s	Hash (FIPS 205)	Different math. Slow but conservative.
Ed25519	Classical EC	Widely tested. Hedge for "old" use cases, not a new quantum-resistant.

If ML-DSA falls to a 2028 cryptanalysis paper, SLH-DSA still authenticates the order. If both lattice and hash fall, the classical signature carries it

THE PROVENANCE LAYER

A hash chain the vault refuses to disobey.

1. Agent builds order. PQ-signs it.
2. `AuditAnchor.anchor(orderHash)` on Monad. Links to caller's previous hash.
3. `RoutingVault.executeAndRoute(orderHash, ...)` refuses execution unless `ANCHOR.lastHash[msg.sender] == orderHash`.

WHAT THIS KILLS

- Replay (sequence enforced per wallet)
- Off-chain order tampering (hash doesn't match)
- Vault impersonation (pair

PROOF, NOT PROMISES

**6 contracts. 84 tests.
Live on Monad testnet.**

6

CONTRACTS
MONADSCAN-VERIFIED

84

TESTS PASSING
PYTHON + FOUNDRY

3

SIGNATURE ALGORITHMS
PER ORDER, HEDGED

AuditAnchor	0x0e649C383CFA6be1998445D0A7a8E1cc7540D239
RoutingVault	0x70580f77d7602f9a03fd34f17f3cc395bbce6938

THE END-TO-END DEMO

One MON in. Two PQ-signed, anchor-gated swaps out.

STEP	CONTRACT	WHAT HAPPENED
	AuditAnchor	0xca148bff...581b
2. Execute	RoutingVault	0.1 MON → 117.52 mUSDC + 117.52 mUSDT , 50/50, single TX

Sandwich-resistant by design: caller-supplied `amountOutMin` flows straight to the pair. No on-chain quote step to race.

WHY US

We ship the part everyone else handwaves.

- **Most PQ-DeFi pitches:** slides about a future migration.
- **Most QAOA-finance pitches:** one hand-picked seed on a simulator.
- **Most provenance pitches:** off-chain Merkle trees nobody verifies.
- **Ours:** verified contracts, real QPU runs with statistical rigor, on-chain hash chain enforced by the vault itself.
- One repo. One CI. Reproducible from a cold clone.

WHAT'S NEXT

From testnet rigor to institutional pilot.

Q3 2026

- Schema-version address-binding
- Capability registry (signature $\neq$ capability)
- Monad mainnet deploy

Q4 2026

- Hot-path PQ cosigner service
- Pilot: Q-Day-resistant treasury rail
- Independent third-party audit

THE ASK

Pilot with Santander.

**Q-Day-resistant crypto exposure for
institutional treasury,
on rails we can prove are honest.**

github.com/EmpowerTours/quantum-portfolio · commit [b3d8166](#)